IFCE | CAMPUS BOA VIAGEM

Tecnologia em Análise e Desenvolvimento de Sistemas
Programação Web I

Estacionamento Central

Sistema web orientado a objetos em PHP

Identificação	Informação
Autor	Luiz Henrique de Sousa Vieira
Professor	Prof. Dr. Renato William Rodrigues de Souza
Tema	08 - Estacionamento
Natureza	Trabalho individual de Programação Web I
Data	03 de novembro de 2026

Relatório de desenvolvimento

Apresento o problema, os requisitos, a modelagem, a implementação orientada a objetos, a camada HTTP, os testes e o uso de inteligência artificial durante o desenvolvimento do projeto.

Repositório: github.com/LuizHenrique704/pweb1-trabalho-oo

Boa Viagem - CE
2026

1. Introdução

Desenvolvi o Estacionamento Central para representar o registro de estadias em um pequeno estacionamento. O público considerado é o responsável pelo atendimento, que precisa consultar a ocupação, registrar um veículo e calcular o valor correspondente à duração informada. O projeto integra requisições HTTP, PHP e orientação a objetos em uma aplicação de escopo reduzido.

O sistema recebe carros e motos, aplica tarifas distintas e apresenta lista, detalhes e resumo das estadias. A duração é informada em horas inteiras. Cada registro ocupa uma vaga; não implementei encerramento, liberação de vagas, pagamentos ou cálculo automático entre horários. O total representa a soma dos valores registrados, e não uma confirmação de recebimento financeiro.

2. Requisitos

Funcionalidade	Comportamento
Consulta	Visão geral, lista e detalhe por identificador.
Cadastro	Formulário POST com mensagens das exceções.
HTTP e JSON	Rotas por query string, status 404 e saída JSON.
Persistência	Gravação e reconstrução dos objetos em arquivo JSON.

Regra	Responsável
Placa não vazia, inclusive após retirar espaços	Veiculo::__construct()
Duração e identificador maiores que zero	Estadia::__construct()
Nome não vazio e capacidade maior que zero	Estacionamento::__construct()
Identificador não repetido e respeito à capacidade	Estacionamento::adicionarEstadia()

Adotei R$ 5,00 por hora para carros e R$ 3,00 para motos. O cálculo é horas × tarifa. A camada web também verifica se o tipo é permitido e se a duração é inteira. As sete regras do domínio são recusadas por exceções, sem depender apenas dos controles do navegador.

3. Modelagem

Organizei o domínio em cinco classes. O diagrama sintetiza as relações e os principais membros; o detalhamento em Mermaid, incluindo construtores e getters, está em docs/modelagem.md. O sinal - indica atributo privado e + indica método público.

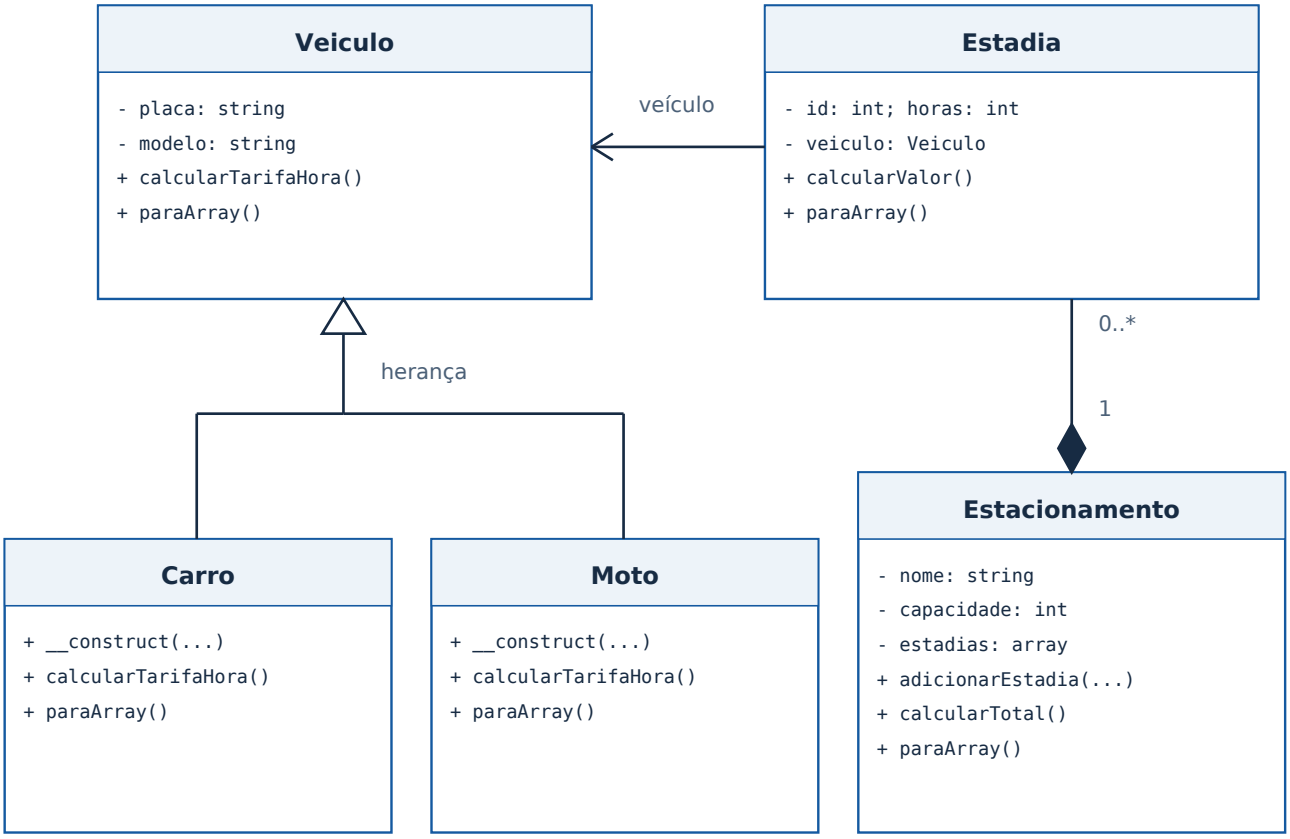


Figura 1. Diagrama de classes do código entregue. Triângulo vazio: herança; losango preenchido: composição; seta simples: associação.

Herança e associação

Carro e Moto são especializações de Veiculo, compartilhando placa, modelo e validação. Cada filha chama parent::__construct() e sobrescreve a tarifa e a conversão para array. Estadia referencia um Veiculo, permitindo receber as duas especializações sem escolher a tarifa por condicionais.

Composição e encapsulamento

Estacionamento mantém a lista de Estadia, controla inclusões e soma os valores. No escopo adotado, as estadias pertencem ao estacionamento; a relação é representada como composição. O PHP não impõe destruição automática das referências externas. Todos os atributos são privados e não há setters: a inclusão passa por adicionarEstadia(), preservando as regras da lista.

4. Implementação orientada a objetos

OO1 e OO4 - Tipos, construtores e encapsulamento

As cinco classes possuem construtor, atributos tipados e declare(strict_types=1) em seus arquivos. Os atributos são privados. A declaração strict_types atua nas chamadas originadas no arquivo e não substitui as validações de negócio. Os getters permitem consulta sem oferecer alteração direta dos atributos.

```
declare(strict_types=1);

class Veiculo
{
    private string $placa;
    private string $modelo;

    public function __construct(string $placa, string $modelo)
    {
        $placa = strtoupper(trim($placa));
        if ($placa === '') {
            throw new Exception('A placa não pode estar vazia.');
        }
        $this->placa = $placa;
        $this->modelo = trim($modelo);
    }
}
```

Trecho de src/Veiculo.php; linhas em branco condensadas para apresentação.

OO2 - Hierarquia, parent e sobrescrita

```
class Moto extends Veiculo
{
    public function __construct(string $placa, string $modelo)
    {
        parent::__construct($placa, $modelo);
    }

    public function calcularTarifaHora(): float
    {
        return 3.0;
    }
}
```

Carro tem estrutura equivalente e retorna 5.0. Veiculo também possui tarifa padrão de 5.0; a sobrescrita em Carro torna explícita a tarifa da especialização. Não utilizei interfaces ou classes abstratas, pois não eram obrigatórias e a hierarquia concreta atende ao escopo.

OO3 - Chamada polimórfica

```
$veiculos = [$carro, $moto];
$tarifasEsperadas = [5.0, 3.0];

foreach ($veiculos as $indice => $objeto) {
    $tarifa = $objeto->calcularTarifaHora();
```

Esse trecho de teste.php percorre objetos de classes diferentes e chama o mesmo método. A verificação seguinte compara o resultado à tarifa esperada. Em Estadia, a mesma escolha dinâmica ocorre no cálculo do valor.

4. Implementação - continuação

OO5 - Exceções e tratamento

Implementei sete recusas de domínio. No exemplo abaixo, Estadia impede duração não positiva. O formulário captura Exception, exibe a mensagem com escape HTML e responde com status 422. O teste também captura a exceção e confere sua mensagem.

```
// src/Estadia.php
if ($horas <= 0) {
    throw new Exception(
        'A duração da estadia deve ser maior que zero.'
    );
}

// public/index.php: tratamento do formulário
} catch (Exception $e) {
    $erro = $e->getMessage();
    http_response_code(422);
}
```

OO6 - Lista de objetos e cálculo

```
// src/Estacionamento.php: antes da inclusão
if (count($this->estadias) >= $this->capacidade) {
    throw new Exception('O estacionamento está lotado.');
}
$this->estadias[] = $estadia;

public function calcularTotal(): float
{
    $total = 0.0;
    foreach ($this->estadias as $estadia) {
        $total += $estadia->calcularValor();
    }
    return $total;
}
```

O método de inclusão também recusa identificador repetido antes de alterar a lista. Estadia calcula horas multiplicadas por calcularTarifaHora() do veículo. No cenário dos testes, 3 × R$ 5,00 + 2 × R$ 3,00 resulta em R$ 21,00.

OO7 - Conversão das relações para array

```
// src/Estadia.php: retorno de paraArray()
return [
    'id' => $this->id,
    'veiculo' => $this->veiculo->paraArray(),
    'horas' => $this->horas,
    'valor' => $this->calcularValor(),
];
```

Todas as cinco classes possuem paraArray(). Carro e Moto acrescentam o campo tipo ao array da mãe; Estacionamento percorre as estadias e converte cada uma. Assim, as relações ficam explícitas antes de json_encode().

OO8 - Organização e carregamento

Mantive uma classe por arquivo em src/. Os carregamentos utilizam require_once com __DIR__, como em require_once __DIR__ . '/../src/Carro.php'; no ponto de entrada. As classes dependentes também carregam suas dependências por caminhos relativos ao próprio arquivo.

5. Camada web

O arquivo public/index.php concentra o processamento antes do HTML: carrega os dados, seleciona a página, trata POST, busca o detalhe e responde em JSON quando solicitado. O HTML posterior contém laços, condicionais e expressões de apresentação. Cabeçalho e menu são compartilhados pelas páginas HTML, inclusive a resposta 404.

Rota / ação	Método	Status
/ ou ?pagina=inicio	GET	200
?pagina=estadias	GET	200
?pagina=detalhe&id=1, se existir	GET	200
?pagina=cadastro	GET	200
?pagina=cadastro, dados válidos	POST	303
?pagina=cadastro, dados recusados	POST	422
?formato=json	GET	200
Página ou identificador inexistente	GET	404
Falha de leitura ou gravação	GET/POST	500

Formulário, validação e redirecionamento

O formulário envia tipo, placa, modelo e horas. Valido a duração com FILTER_VALIDATE_INT e crio o veículo da especialização escolhida. O próximo identificador é o maior existente mais um. Após adicionar e salvar a estadia, a resposta 303 direciona para o detalhe por GET, evitando repetir o POST ao atualizar a página.

```
header(
    'Location: ?pagina=detalhe&id=' . $novaEstadia->getId(),
    true,
    303
);
exit;
```

Evidências HTTP fornecidas pelo ambiente local

Os cabeçalhos abaixo foram registrados em docs/testes.md. No Linux, o executável é curl; o enunciado usa curl.exe. Mantive os valores registrados, incluindo datas e versão do PHP. A evidência 200 não identifica a página no comando de exemplo original, mas comprova uma resposta HTML bem-sucedida.

```
HTTP/1.1 200 OK
Host: localhost:8000
Date: Tue, 03 Nov 2026 10:53:40 GMT
Connection: close
X-Powered-By: PHP/8.5.12
Content-Type: text/html; charset=utf-8

HTTP/1.1 404 Not Found
Host: localhost:8000
Date: Tue, 03 Nov 2026 10:54:12 GMT
Connection: close
X-Powered-By: PHP/8.5.12
Content-Type: text/html; charset=utf-8
```

5. Camada web - JSON e persistência

W2 - Resposta JSON

```
curl -i "http://localhost:8000/?formato=json"

HTTP/1.1 200 OK
Host: localhost:8000
Date: Tue, 03 Nov 2026 10:58:32 GMT
Connection: close
X-Powered-By: PHP/8.5.12
Content-Type: application/json; charset=utf-8
```

A rota usa paraArray(), json_encode(), JSON_UNESCAPED_UNICODE e JSON_PRETTY_PRINT. O exit impede anexar HTML ao JSON. Os cabeçalhos acima são transcritos do registro; o corpo da resposta não foi incluído nesse registro. O roteiro do núcleo verifica separadamente a conversão e a leitura do JSON.

W3 e W4 - Recusa e escape HTML

```
HTTP/1.1 422 Unknown Status Code
Host: localhost:8000
Date: Tue, 03 Nov 2026 11:00:28 GMT
Connection: close
X-Powered-By: PHP/8.5.12
Content-Type: text/html; charset=utf-8
```

O código numérico observado é 422; a expressão textual acima é a enviada pelo servidor registrado. Para a exibição segura, escapar() chama htmlspecialchars com ENT_QUOTES, ENT_SUBSTITUTE e UTF-8. Informei Teste como modelo e as tags apareceram literalmente, conforme o registro de testes.

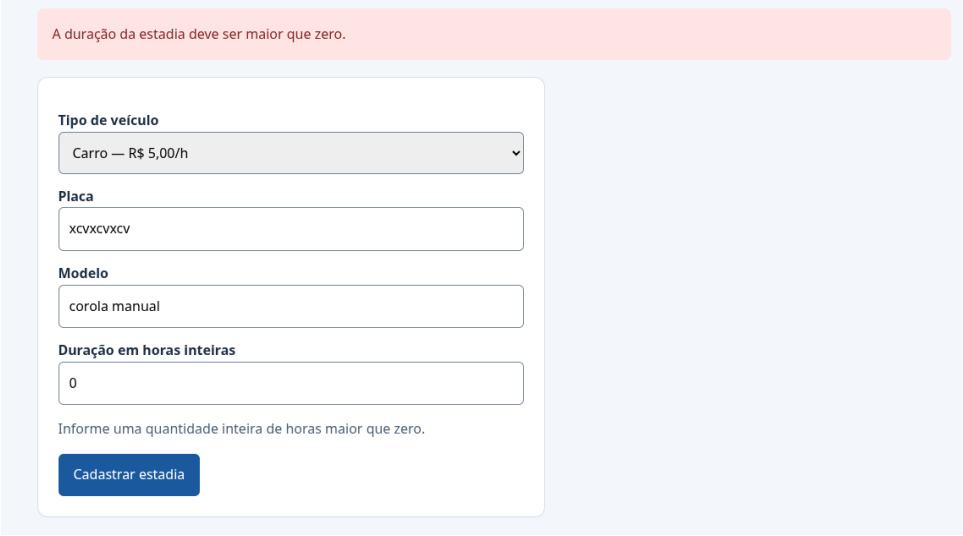


Figura 2. Mensagem de recusa da duração no formulário. Fonte: docs/formulario-invalido.png.

Persistência em arquivo

salvarEstacionamento() grava com file_put_contents e LOCK_EX; carregarEstacionamento() lê com file_get_contents e json_decode e reconstrói as classes pelos construtores. Os exemplos são criados apenas quando o arquivo não existe. O caminho dados/estacionamento.json fica fora de public/. O registro de testes informa manutenção dos dados após atualização e reinício do servidor e ausência de inclusão com duração zero.

LOCK_EX protege a gravação; não torna todo o ciclo de leitura, alteração e escrita uma transação. Para uso simultâneo em produção, seria necessário ampliar esse controle. Esta implementação foi delimitada à demonstração local.

6. Testes - veículos e estadias

As saídas das páginas 8 e 9 são a transcrição do roteiro final registrado em docs/testes.md, sem os testes avulsos anteriores. O ambiente informado pelo autor é Linux com PHP 8.5.12. A execução foi realizada pelo autor; não apresento estes registros como uma nova execução independente.

```
php -l teste.php
No syntax errors detected in teste.php
php teste.php

=== 1. Veículos e encapsulamento ===
OK: Veiculo aceita dados válidos e aplica a tarifa padrão.
OK: Getters retornam placas e modelos padronizados.
OK: Veiculo recusa placa '' — A placa não pode estar vazia.
OK: Veiculo recusa placa '   ' — A placa não pode estar vazia.
OK: Carro recusa placa '' — A placa não pode estar vazia.
OK: Carro recusa placa '   ' — A placa não pode estar vazia.
OK: Moto recusa placa '' — A placa não pode estar vazia.
OK: Moto recusa placa '   ' — A placa não pode estar vazia.

=== 2. Herança e polimorfismo ===
OK: Carro responde ao mesmo método com tarifa de R$ 5,00
OK: Moto responde ao mesmo método com tarifa de R$ 3,00

=== 3. Estadias e cálculo individual ===
OK: Estadia guarda identificador, veículo e horas.
OK: Carro por 3 horas custa R$ 15,00; moto por 2 horas custa R$ 6,00.
OK: Recusa duração de 0 horas — A duração da estadia deve ser maior que zero.
OK: Recusa duração de -2 horas — A duração da estadia deve ser maior que zero.
OK: Recusa identificador 0 — O identificador da estadia deve ser maior que zero.
OK: Recusa identificador -1 — O identificador da estadia deve ser maior que zero.
```

O que as verificações comprovam

O grupo 1 verifica o objeto base, os getters e a normalização de placa e modelo. As seis recusas cobrem placa vazia e somente espaços em Veiculo, Carro e Moto, demonstrando também que as filhas reaproveitam a validação da mãe.

O grupo 2 percorre carro e moto no mesmo foreach, verifica a relação instanceof Veiculo e compara cada tarifa com o valor esperado. O grupo 3 confere os dados de Estadia, os cálculos de R$ 15,00 e R$ 6,00 e as recusas de duração e identificador não positivos.

6. Testes - composição e JSON

```
=== 4. Estacionamento e composição ===
OK: Estacionamento válido começa vazio e com total zero.
OK: Aceita a primeira estadia enquanto há vaga.
OK: Recusa identificador repetido mesmo havendo vaga — Já existe uma estadia com esse
  identificador.
OK: Recusa por identificador repetido preserva os dados.
OK: Aceita identificador diferente na última vaga e soma R$ 21,00.
OK: Recusa nova estadia quando a capacidade foi atingida — O estacionamento está lotado.
OK: Recusa por lotação preserva os dados.
OK: Recusa nome '' — O nome do estacionamento não pode estar vazio.
OK: Recusa nome '   ' — O nome do estacionamento não pode estar vazio.
OK: Recusa capacidade 0 — A capacidade deve ser maior que zero.
OK: Recusa capacidade -1 — A capacidade deve ser maior que zero.

=== 5. Conversão para array e JSON ===
OK: Veiculo converte seus dados para array.
OK: Classes filhas incluem tipo e tarifa correta no array.
OK: Composição inteira é convertida em arrays com os valores esperados.
OK: JSON gerado pode ser lido preservando tipo do veículo e total.

SUCESSO: todos os testes do núcleo passaram.
```

Interpretação dos resultados

O grupo 4 demonstra estacionamento válido, estado inicial vazio e total zero. A primeira estadia é aceita; a tentativa de repetir seu identificador é recusada mesmo havendo vaga. A segunda estadia tem identificador diferente, ocupa a última vaga e leva o total a R$ 21,00. A terceira é recusada por lotação. As comparações de arrays antes e depois das recusas comprovam que a lista e os valores não foram alterados.

As demais recusas do grupo 4 verificam nome vazio, nome com espaços e capacidades zero e negativa. O grupo 5 confere arrays da mãe e das filhas, toda a composição e a leitura do JSON gerado. A mensagem final aparece apenas se todas as verificações forem aprovadas.

Mecanismo do roteiro e alcance

verificar() encerra com exit(1) quando a condição não é atendida. testarRecusa() executa uma função recebida, captura Exception e compara sua mensagem com a esperada; aceitar uma operação inválida também encerra com falha. Esses testes verificam o núcleo em memória; as evidências HTTP, a persistência e o escape são complementados pelos testes manuais documentados na seção anterior.

7. Ferramentas utilizadas

Ferramenta	Papel no desenvolvimento
PHP 8.5.12 (ambiente informado)	Classes, requisições, exceções, JSON e servidor local.
Linux e terminal	Execução do PHP, comandos de Git e verificação HTTP.
VS Code	Edição das classes, ponto de entrada e documentação.
Git / GitHub Desktop	Branches, commits, integração e envio ao remoto.
GitHub	Repositório, issues, pull requests e tags dos marcos.
curl	Inspeção de status e cabeçalhos HTTP registrados.
Navegador e DevTools	Navegação e inspeção da interface e das requisições.
ChatGPT	Apoio à interpretação, implementação, revisão e explicação.
Gravador de tela	Ferramenta prevista para o vídeo; o nome não está registrado nos arquivos enviados.

O roteiro de execução utiliza php -S localhost:8000 -t public -d display_errors=1 -d output_buffering=0. O -t delimita a pasta pública. As opções de erro são adequadas à demonstração de desenvolvimento. O projeto não depende de Composer nem de banco de dados.

8. Uso de LLM

Registrei cinco interações com ChatGPT em docs/llm.md, datadas de 02/11/2026. Cada uma contém prompt, resumo e decisão sobre a resposta. O conteúdo trata da interpretação do trabalho, diagnóstico de arquivo ausente, criação da tag, consolidação do index.php e exibição segura de HTML.

Interação	Contribuição e decisão registrada
1 - Planejamento	Organização das exigências e adoção de uma sequência de trabalho.
2 - Arquivo ausente	Conferência de caminhos, nomes e branches antes de mudar a lógica.
3 - Tag	Substituição de uma orientação de interface que não consegui localizar por git tag no terminal.
4 - index.php completo	Consolidação dos trechos e manutenção do processamento antes do HTML.
5 - Escape HTML	Manutenção de htmlspecialchars para exibir as tags como texto.

A sugestão inadequada documentada foi um caminho da interface do GitHub Desktop que não consegui utilizar. Informei a dificuldade e adotei a criação local da tag pelo terminal. Esse episódio mostrou a necessidade de conferir a resposta no ambiente real. As mensagens Assistido-por: ChatGPT são a forma prevista para identificar ajuda relevante nos commits; o ZIP não contém o histórico Git para uma auditoria dessas mensagens.

9. Dificuldades, aprendizados e conclusão

Dificuldades e decisões

Durante os testes, encontrei erros de require_once por arquivos não disponíveis no caminho esperado. A resolução envolveu conferir nomes, arquivos e branches. Aprendi a distinguir uma falha de carregamento de uma falha na regra de negócio: se a classe não é encontrada, a validação nem chega a ser executada.

Outra dificuldade foi entender por que os cadastros desapareciam na primeira versão. Os objetos eram recriados em cada requisição. Acrescentei a persistência em JSON para reconstruir o estado nos próximos acessos e preservei o processamento antes da saída HTML. O redirecionamento 303 separou o cadastro POST da consulta GET.

Também foi necessário adaptar o fluxo Git ao meu ambiente. Utilizei GitHub Desktop para as operações autenticadas e o terminal para comandos locais e testes. O uso de LLM facilitou o avanço por etapas, mas exigiu comparação com as mensagens reais e com as opções disponíveis na instalação.

Aprendizados

Compreendi a herança ao reaproveitar a validação de Veiculo nas filhas e o polimorfismo ao obter tarifas distintas pela mesma chamada. A composição permitiu manter a lista de estadias e centralizar a capacidade e o total. As exceções impediram a criação ou inclusão de estados inválidos, enquanto os testes verificaram também a preservação dos dados após recusas.

A camada web mostrou que a resposta inclui conteúdo e status HTTP. Diferenciei sucesso, redirecionamento, recusa e recurso inexistente, além de separar JSON de HTML. O teste com tags no modelo mostrou por que o conteúdo recebido precisa passar por escape antes da exibição.

Limitações e próximos avanços

O escopo atual mantém cada registro como vaga ocupada. Evoluções possíveis são encerramento de estadias, histórico separado da ocupação e cálculo entre horários. A persistência simples também poderia ser substituída por armazenamento transacional para uso simultâneo. Essas funções não são apresentadas como implementadas.

Na revisão do pacote, o arquivo dados/estacionamento.json estava incluído apesar da regra no .gitignore. Essa regra não remove arquivos já rastreados: a distribuição ainda precisa retirar o arquivo do índice para que novos clones iniciem com os exemplos. As evidências web fornecidas trazem os cabeçalhos, mas não o log do servidor nem uma saída específica para detalhe inexistente. São complementos documentais a concluir; não foram inventados neste relatório.

Conclusão

O projeto reúne as cinco classes, as regras de negócio, o formulário, as consultas e a persistência em uma aplicação pequena. Os resultados registrados aprovam os testes do núcleo e demonstram respostas HTTP 200, 404 e 422. O desenvolvimento permitiu relacionar o modelo de objetos ao ciclo de requisição e resposta e reconhecer que a documentação e a verificação fazem parte da entrega.

Base documental: enunciado trabalho-oo-php.pdf; código e documentos do arquivo pweb1-trabalho-oo-main.zip (identificador do snapshot: 34fc7cdeac25246763950bb4985ca54a29b1779a). Relatório preparado em 03/11/2026. O vídeo ainda consta como pendente no README do snapshot.